

AENEXZ PROJECT REPORT

PERFORMANCE COMPARISON OF MACHINE LEARNING ALGORITHMS(LOGISTIC REGRESSION,RANDOM FOREST ,SVM) ON A HEART DISEASE PREDICTION

Internship Project Report

Submitted in partial fulfillment of the internship
requirements in Data Analytics

Submitted To

AEXON

Submitted By

VAGARE SNEHAL BIRUDEV

Data Analytics Intern

Index

Sr. No.	Contents
1	Abstract
2	Introduction
3	Literature Review
4	Methodology
4.1	Dataset Description
4.2	Data Preprocessing
4.2.1	Handling Categorical Features
4.2.2	One-Hot Encoding
4.2.3	Feature Alignment
4.3	Machine Learning Algorithms
4.3.1	Logistic Regression
4.3.2	Support Vector Machine (SVM)
4.3.3	Random Forest
4.4	Hyperparameter Tuning
4.4.1	Logistic Regression Parameters
4.4.2	SVM Parameters
4.4.3	Random Forest Parameters
5	Experimental Setup
5.1	Software and Tools Used
5.2	Dataset Loading
5.3	Initial Dataset Analysis
5.4	Missing Value Detection
5.5	Duplicate Value Removal

5.6	Feature Scaling
5.7	Outlier Detection and Handling
5.8	Feature Selection and Target Separation
5.9	Splitting the Dataset
5.10	Model Training
5.10.1	Logistic Regression Training
5.10.2	Random Forest Training
5.10.3	SVM Training
5.11	Hyperparameter Tuning using GridSearchCV
5.12	Model Prediction
5.13	Performance Evaluation Metrics
5.13.1	Accuracy
5.13.2	Precision
5.13.3	Recall
5.13.4	F1-Score
5.13.5	Confusion Matrix
5.14	Data Visualization
6	Results and Analysis
6.1	Experimental Results
6.2	Logistic Regression Results
6.3	Random Forest Results
6.4	Support Vector Machine Results
6.5	Performance Comparison Table
6.6	Training and Testing Accuracy Comparison
6.7	Hyperparameter Tuning Results

6.8	Confusion Matrix Analysis
6.9	Visualization and Graphical Analysis
6.10	Comparative Analysis of Models
7	Conclusion and Future Scope
7.1	Conclusion
7.2	Future Scope
8	References

Abstract

Heart disease datasets are commonly used to evaluate the performance of machine learning algorithms in classification problems. This project focuses on the comparative analysis of different machine learning algorithms to study their performance on a given dataset. The dataset consists of medical attributes such as age, sex, chest pain type, resting blood pressure, cholesterol level, fasting blood sugar, resting ECG results, maximum heart rate, exercise-induced angina, oldpeak, slope, number of vessels colored by fluoroscopy, and thalassemia.

Data preprocessing techniques, including one-hot encoding, were applied to prepare the dataset for analysis. Machine learning algorithms such as Logistic Regression, Support Vector Machine (SVM), and Random Forest were implemented and evaluated. Hyperparameter tuning using GridSearchCV was performed to analyze the effect of different parameter settings on model performance. The evaluation was carried out using metrics such as Accuracy and F1-score. The results show that different algorithms perform differently on the same dataset, and parameter tuning significantly impacts their performance. Among the evaluated models, Random Forest achieved the best overall results in terms of Accuracy and F1-score.

Introduction

Machine learning techniques are widely used for classification and data analysis tasks, where selecting the appropriate algorithm plays a crucial role in achieving better performance. Different algorithms behave differently depending on the dataset and parameter settings, making comparative analysis essential for selecting the most suitable model.

This project focuses on the comparative analysis of machine learning algorithms using a heart disease dataset. The dataset includes several important medical attributes such as age, sex, chest pain type, resting blood pressure, cholesterol level, fasting blood sugar, resting ECG results, maximum heart rate, exercise-induced angina, oldpeak, slope, number of vessels colored by fluoroscopy, and thalassemia. These features play a significant role in analyzing patterns within the dataset.

The main objective of this study is to compare the performance of Logistic Regression, Support Vector Machine (SVM), and Random Forest algorithms and to analyze how changes in their parameters affect performance. Hyperparameter tuning is performed using GridSearchCV to evaluate different parameter combinations.

The models are evaluated using performance metrics such as Accuracy and F1-score. This study helps in understanding how algorithm selection and parameter tuning influence model performance and assists in identifying the most effective algorithm for the given dataset.

Literature Review

Machine learning algorithms are widely used for classification tasks, where the goal is to analyze data and categorize it into different classes. Different algorithms perform differently depending on the dataset and parameter settings, making it important to compare their performance.

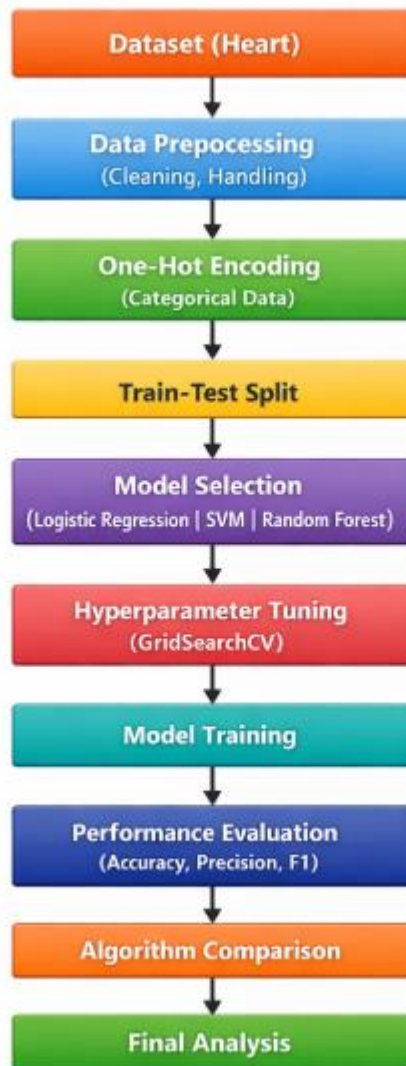
Logistic Regression is a simple and commonly used algorithm for binary classification problems. It works well when the relationship between input features and output is linear. Support Vector Machine (SVM) is another powerful algorithm that is effective in handling complex data by finding optimal decision boundaries. Random Forest is an ensemble learning method that combines multiple decision trees to improve performance and reduce overfitting.

The performance of these algorithms depends not only on the dataset but also on the selection of hyperparameters. Parameters such as regularization in Logistic Regression, kernel settings in SVM, and tree-related parameters in Random Forest can significantly affect model performance.

Therefore, it is important to analyze how different algorithms behave on the same dataset and how changes in parameters impact evaluation metrics such as Accuracy and F1-score. This study focuses on comparing Logistic Regression, SVM, and Random Forest and analyzing the effect of parameter tuning on their performance.

Methodology

Steps :



1. Dataset Description :

The dataset used in this study is a heart disease dataset that contains medical information related to patients. It is commonly used for classification and analysis of heart-related conditions. The dataset consists of 1025 instances and 14 features, where each instance represents an individual patient.

The features include age, sex, chest pain type, resting blood pressure, cholesterol level, fasting blood sugar, resting electrocardiographic (ECG) results, maximum heart rate, exercise-induced angina, oldpeak, slope, number of vessels colored by fluoroscopy, and thalassemia. The dataset contains both numerical and categorical attributes.

In addition to these input features, the dataset includes a target variable that indicates the presence or absence of heart disease. The target variable is binary in nature, where 0 represents the absence and 1 represents the presence of heart disease.

This dataset is used in the study to perform a comparative analysis of different machine learning algorithms and to analyze how parameter tuning affects performance metrics such as Accuracy and F1-score.

Data Preprocessing :

Data preprocessing was performed to convert the raw dataset into a structured format suitable for machine learning algorithms. This step is essential because raw data often contains categorical values and inconsistencies that cannot be directly processed by machine learning models. Proper preprocessing improves data quality and helps in achieving better model performance.

The preprocessing steps included handling categorical variables, splitting the data into training and testing sets, and ensuring consistency between both datasets.

Initially, categorical features were identified using the following function:

```
categorical_cols = X_train.select_dtypes(include=['object']).columns
```

The `select_dtypes()` function is used to select columns based on their data types. The parameter `include=['object']` specifies that only columns with object data type (categorical features) should be selected. This function works by scanning the dataset and filtering out columns that match the specified data type. The `.columns` attribute is then used to extract the names of these columns for further processing.

To transform these categorical features into numerical form, one-hot encoding was applied using:

```
X_train = pd.get_dummies(X_train, columns=categorical_cols, drop_first=True)
X_test = pd.get_dummies(X_test, columns=categorical_cols, drop_first=True)
```

The `get_dummies()` function converts categorical variables into multiple binary columns. The parameter `columns=categorical_cols` specifies which columns should be encoded. The parameter `drop_first=True` removes the first category from each feature to avoid redundancy and prevent multicollinearity. Internally, this function creates new columns for each category and assigns a value of 1 if the category is present and 0 otherwise.

After encoding, differences in feature columns may arise between the training and testing datasets due to variations in categorical values. To resolve this, feature alignment was performed using:

```
X_train, X_test = X_train.align(X_test, join='outer', axis=1, fill_value=0)
```

The `align()` function is used to synchronize the structure of two datasets. The parameter `join='outer'` ensures that all columns from both datasets are included. The parameter `axis=1` specifies that alignment should be performed column-wise. The parameter `fill_value=0` replaces missing values with zero. This

function works by comparing the columns of both datasets and adding any missing columns where required, ensuring both datasets have identical structures.

Overall, these preprocessing steps are necessary to ensure that the data is clean, consistent, and in a format suitable for machine learning models. Proper preprocessing directly impacts the accuracy, stability, and reliability of the models used in this analysis. The same preprocessing pipeline was applied across all models to ensure a fair and consistent comparison.

Algorithm :

After preprocessing the dataset, three machine learning algorithms were applied for comparative analysis: Logistic Regression, Support Vector Machine (SVM), and Random Forest. These algorithms were selected to evaluate their performance and to analyze how different models behave on the same dataset.

Logistic Regression is a linear classification algorithm used for binary classification problems. It works by applying a sigmoid function to estimate the probability of a class and learns a linear relationship between input features and the target variable. The parameter `C` controls regularization strength, where smaller values increase regularization and help prevent overfitting. The penalty (L1 or L2) affects feature selection and model complexity, while `class_weight` helps handle imbalanced datasets.

Support Vector Machine (SVM) is a classification algorithm that finds the optimal hyperplane to separate classes by maximizing the margin between data points. It can handle both linear and non-linear data using kernel functions. The parameter `C` controls the trade-off between maximizing margin and minimizing classification error. The kernel parameter defines the type of decision boundary, and `gamma` determines how far the influence of a single data point reaches, affecting model flexibility.

Random Forest is an ensemble learning method that builds multiple decision trees and combines their outputs to improve prediction accuracy and reduce overfitting. It works by training each tree on a random subset of data and features. The parameter `n_estimators` controls the number of trees, `max_depth` limits tree growth to avoid overfitting, and `min_samples_split` determines the minimum samples required to split a node, affecting model generalization.

All models were trained on the same preprocessed dataset to ensure a fair comparison. The detailed tuning of these parameters is discussed in the Hyperparameter Tuning section.

Hyperparameter :

In machine learning, hyperparameters are the configuration settings that are defined before the training process begins. Unlike model parameters, which are learned automatically from the data during training, hyperparameters are manually set values that control how the model learns from the data. These settings define the learning behavior, complexity, and performance of the model.

Some important concepts used in this context include training data (data used to train the model), testing data (data used to evaluate model performance), overfitting (when the model memorizes training data instead of learning patterns), and underfitting (when the model fails to learn important patterns). These concepts are strongly influenced by the choice of hyperparameters because they control how flexible or strict the learning process becomes.

In this project, the dataset is first divided into training and testing sets using the `train_test_split` function. This step is important because it ensures that the model is trained on one portion of the data and evaluated on unseen data, which gives a realistic measure of performance. The implementation is as follows:

```
X_train,X_test,y_train,y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Here, the `test_size` parameter defines the proportion of data used for testing, and `random_state` ensures that the same split is obtained every time for reproducibility.

After splitting the data, hyperparameter tuning is performed using `GridSearchCV`. This method systematically tries all possible combinations of hyperparameters and selects the best model based on cross-validation performance. It reduces manual effort and ensures optimal selection of parameters. The implementation is:

```
grid = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid.fit(X_train, y_train)
```

In this, `cv=5` means that the training data is divided into five parts, and the model is trained and validated multiple times to ensure stable and reliable results. The scoring parameter is set to accuracy, which means the best model is selected based on highest accuracy.

Hyperparameters control different aspects of the learning process such as model complexity, decision boundaries, and sensitivity to data patterns. Different algorithms use different hyperparameters because each algorithm follows a different mathematical approach to learning.

For Logistic Regression, important hyperparameters include `C`, `penalty`, `class_weight`, and `max_iter`. The implementation used in this project is:

```
model = LogisticRegression()
```

```
param_grid =
```

```
{'C': [0.01, 0.1, 1, 10],
```

```
'penalty': ['l1', 'l2'],
```

```
'class_weight': [None, 'balanced']},
```

```
'max_iter': [100, 200]}
```

The parameter C controls the strength of regularization, which is used to prevent overfitting by penalizing large coefficients. A smaller value of C makes the model simpler and more generalized, while a larger value allows the model to fit the training data more closely, which may improve accuracy but also increase the risk of overfitting. The penalty parameter defines the type of regularization used, where L1 helps in feature selection by eliminating less important features, while L2 helps in stabilizing the model by reducing large weight values. The class_weight parameter is used when the dataset is imbalanced, ensuring that minority classes are given more importance. The max_iter parameter defines the maximum number of iterations allowed for optimization so that the model can converge properly during training.

For Support Vector Machine (SVM), important hyperparameters include C, kernel, and gamma. The implementation is:

```
model = SVC()
```

```
param_grid = {
```

```
'C': [0.1, 1, 10],
```

```
'kernel': ['linear', 'rbf'],
```

```
'gamma': ['scale', 'auto']}]
```

The parameter C controls the balance between maximizing the margin and minimizing classification errors. A higher value of C makes the model focus more on correctly classifying training data, which may lead to overfitting, while a lower value allows a wider margin and better generalization. The kernel parameter defines how the input data is transformed into higher dimensions to make it separable; linear kernel is used for simple datasets, while RBF kernel is used for complex non-linear patterns. The gamma parameter controls how far the influence of a single data point reaches; higher gamma values create more complex decision boundaries, while lower values produce smoother and more generalized boundaries.

For Random Forest, important hyperparameters include n_estimators, max_depth, min_samples_split, and min_samples_leaf. The implementation is:

```
model = RandomForestClassifier()
```

```
param_grid = {
```

```
'n_estimators': [50, 100, 200],
```

```
'max_depth': [None, 10, 20]
```

```
'min_samples_split': [2, 5]
```

```
'min_samples_leaf': [1, 2]}
```

The parameter `n_estimators` defines the number of decision trees used in the model. Increasing the number of trees improves accuracy and stability because predictions are averaged across multiple models, but it also increases computation time. The `max_depth` parameter controls how deep each decision tree can grow; deeper trees capture more complex patterns but may lead to overfitting, while limited depth improves generalization. The `min_samples_split` parameter defines the minimum number of samples required to split a node, helping to control model complexity. The `min_samples_leaf` parameter ensures that each leaf node has a minimum number of samples, which prevents overfitting and improves model stability.

Overall, hyperparameters play a very important role in machine learning models because they directly control how the model learns from data. They significantly affect accuracy, complexity, and generalization ability. In this project, hyperparameter tuning using `GridSearchCV` ensures that the best combination of parameters is selected for Logistic Regression, SVM, and Random Forest models, resulting in improved performance and reliable predictions on unseen data.

Experimental Setup

The implementation of this project was carried out using the Python programming language in the Google Colab environment. Google Colab is a cloud-based platform that provides a Jupyter Notebook-style interface for writing and executing Python programs without requiring software installation on a local system. It supports machine learning libraries and provides computational resources that simplify data preprocessing, visualization, model implementation, and evaluation. Google Colab was selected because it allows easy execution of machine learning workflows and provides an organized environment for experimentation.

Several Python libraries were used throughout the implementation of this project. These libraries were used for dataset handling, preprocessing, visualization, machine learning model implementation, hyperparameter tuning, and performance evaluation.

The Pandas library was used for loading and managing the dataset. The dataset was imported using the `read_csv()` function:

```
heart_data = pd.read_csv('/content/heart.csv')
```

The `read_csv()` function reads the CSV dataset file and converts it into a Pandas `DataFrame`. A `DataFrame` is a tabular data structure that allows efficient data analysis and manipulation.

Initial dataset analysis was performed using the following functions:

```
heart_data.head()
heart_data.info()
heart_data.describe()
```

The head() function displays the first few rows of the dataset and helps in understanding the structure of the data. The info() function provides information such as column names, data types, number of non-null values, and memory usage. The describe() function generates statistical summaries including mean, standard deviation, minimum value, maximum value, and quartiles for numerical features.

The dataset was checked for missing values and duplicate records to ensure data quality.

```
heart_data.isnull().sum()
heart_data.duplicated().sum()
```

The isnull().sum() function identifies missing values present in each column of the dataset. Missing values can affect model performance and may require preprocessing before training. The duplicated().sum() function counts duplicate records in the dataset.

Duplicate entries were removed using:

```
heart_data.drop_duplicates(inplace=True)
```

The drop_duplicates() function removes repeated rows from the dataset. Removing duplicate records improves data consistency and prevents redundant information from affecting machine learning model performance.

The dataset was separated into input features and target variables before model training.

```
X = heart_data.drop('target', axis=1)
y = heart_data['target']
```

The variable X contains all input features used for prediction, while y contains the target variable representing the presence or absence of heart disease.

Feature scaling was performed using the MinMaxScaler() function from the Scikit-learn library.

```
scaler = MinMaxScaler()
X = scaler.fit_transform(X)
```

The MinMaxScaler() function normalizes feature values between 0 and 1 using the following formula:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Feature scaling is important because some machine learning algorithms are sensitive to differences in feature magnitudes. Scaling ensures that all features contribute equally during model training and improves convergence speed and prediction stability.

The dataset was divided into training and testing datasets using the `train_test_split()` function.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

The `train_test_split()` function divides the dataset into two parts:

- Training Dataset
- Testing Dataset

The parameter `test_size=0.2` indicates that 20% of the dataset is reserved for testing and 80% is used for training. The parameter `random_state=42` ensures reproducibility by generating the same data split during every execution.

Machine learning models were implemented using Scikit-learn functions.

Logistic Regression was implemented using:

```
lr_model = LogisticRegression()
lr_model.fit(X_train, y_train)
```

The `LogisticRegression()` function creates a Logistic Regression classification model. The `fit()` function trains the model using the training dataset by learning the relationship between input features and target values.

Random Forest was implemented using:

```
rf_model = RandomForestClassifier()
rf_model.fit(X_train, y_train)
```

The `RandomForestClassifier()` function creates an ensemble learning model consisting of multiple decision trees. Each decision tree is trained on random subsets of data, and the final prediction is generated by combining outputs from all trees. This improves prediction accuracy and reduces overfitting.

Hyperparameter tuning was performed using the `GridSearchCV()` function.

```
grid = GridSearchCV(model, param_grid, cv=5)
grid.fit(X_train, y_train)
```

The GridSearchCV() function systematically tests different combinations of hyperparameters and selects the best-performing parameter set based on cross-validation performance.

The parameter cv=5 specifies 5-fold cross-validation. In this method, the training dataset is divided into five equal parts. The model is trained multiple times using different combinations of training and validation sets, which improves reliability and reduces overfitting.

Predictions were generated using the predict() function:

```
y_pred = model.predict(X_test)
```

The predict() function uses the trained model to generate predicted outputs for the testing dataset.

Model performance was evaluated using different evaluation metrics such as Accuracy, Precision, Recall, and F1-Score.

```
accuracy_score(y_test, y_pred)
precision_score(y_test, y_pred)
recall_score(y_test, y_pred)
f1_score(y_test, y_pred)
```

These evaluation functions compare predicted values with actual target values and measure classification performance from different perspectives.

Visualization of results was performed using the Matplotlib library.

```
plt.bar(models, accuracy_scores)
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.title("Model Comparison")
```

The bar() function was used to generate bar graphs for comparing model performance. The xlabel(), ylabel(), and title() functions were used to label and describe the graphs properly.

Overall, the experimental setup provided a structured and efficient environment for dataset preprocessing, feature scaling, machine learning implementation, hyperparameter tuning, visualization, and evaluation of model performance on the heart disease dataset.

1. Problem Definition

The main objective of this experiment is to develop a machine learning-based classification system for predicting heart disease using patient medical data. The system analyzes various health-related parameters and predicts whether a patient is likely to suffer from heart disease.

The experiment focuses on applying different machine learning classification algorithms and comparing their performance using evaluation metrics.

2. Dataset Collection

The Heart Disease dataset was collected and loaded into the Python environment using the Pandas library. The dataset contains several medical attributes related to patient health conditions such as age, cholesterol level, blood pressure, chest pain type, heart rate, and other important parameters.

Code Snippet

```
import pandas as pd

heart_data = pd.read_csv('/content/Heart_Disease_Dataset.csv')
```

The first few records of the dataset were displayed to understand the structure of the data.

Code Snippet

```
heart_data.head()
```

3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed to understand the dataset characteristics, attribute types, and statistical information.

The following operations were carried out:

- Checking dataset dimensions
- Displaying attribute information
- Identifying data types
- Generating statistical summaries

Code Snippet

```
heart_data.shape
heart_data.info()
heart_data.describe()
```

EDA helps in:

- Understanding feature distributions
- Detecting anomalies
- Identifying categorical and numerical features
- Understanding relationships between attributes

4. Data Preprocessing

Data preprocessing was performed to improve data quality and prepare the dataset for machine learning algorithms.

4.1 Missing Value Detection

The dataset was checked for null or missing values.

Code Snippet

```
heart_data.isnull().sum()
```

Handling missing values is important because incomplete data can negatively affect model performance and prediction accuracy.

4.2 Duplicate Value Removal

Duplicate records were identified and removed from the dataset.

Code Snippet

```
heart_data.duplicated().sum()
```

```
heart_data.drop_duplicates(inplace=True)
```

Removing duplicate records helps:

- Reduce bias
- Improve model reliability
- Prevent repeated learning patterns

4.3 Feature Scaling

Feature scaling was performed using the MinMaxScaler technique to normalize the data values between 0 and 1.

Code Snippet

```
from sklearn.preprocessing import MinMaxScaler
```

Feature scaling is necessary because:

- Different attributes have different ranges
- Large-value features may dominate smaller-value features
- Scaling improves convergence speed and model performance

5. Outlier Detection and Handling

Outliers in numerical columns were detected using Quartile and Interquartile Range (IQR) methods.

Code Snippet

```
numeric_cols = heart_data.select_dtypes(include=['number']).columns
```

```
for col in numeric_cols:
```

```
    Q1 = heart_data[col].quantile(0.25)
```

```
    Q3 = heart_data[col].quantile(0.75)
```

```
    IQR = Q3 - Q1
```

Outlier handling improves:

- Model stability
- Prediction accuracy
- Data consistency
- Generalization ability

6. Feature Selection and Target Separation

The dataset was divided into:

- Input Features (X)
- Target Variable (Y)

Input features contain patient medical information, while the target variable represents the presence or absence of heart disease.

This step is essential for supervised machine learning classification.

7. Splitting the Dataset

The dataset was divided into training and testing datasets using the `train_test_split()` function.

Code Snippet

```
from sklearn.model_selection import train_test_split
```

Generally:

- 80% data used for training
- 20% data used for testing

Training data is used for model learning, while testing data is used for evaluating model performance on unseen data.

8. Logistic Regression Classification

Logistic Regression is a supervised machine learning algorithm used for binary classification problems.

In this experiment, Logistic Regression was applied to classify whether a patient has heart disease.

Code Snippet

```
from sklearn.linear_model import LogisticRegression
```

The model learns the relationship between medical features and heart disease prediction.

Advantages of Logistic Regression

- Simple and efficient
- Suitable for binary classification
- Fast training process

- Easy to interpret

9. Hyperparameter Tuning using GridSearchCV

Hyperparameter tuning was performed using GridSearchCV to improve model performance.

Code Snippet

```
from sklearn.model_selection import GridSearchCV
```

Different parameter combinations were tested, including:

- Regularization parameter (C)
- Penalty type
- Maximum iterations
- Class weights

Hyperparameter tuning helps:

- Improve accuracy
- Reduce overfitting
- Find optimal model configuration

10. Random Forest Classification

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve prediction performance.

Code Snippet

```
from sklearn.ensemble import RandomForestClassifier
```

Random Forest helps:

- Reduce overfitting
- Improve accuracy
- Handle non-linear relationships
- Provide feature importance analysis

10.1 Random Forest Parameter Tuning

Different hyperparameters were tested to optimize model performance.

Code Snippet

```
param_grid = {  
    "n_estimators": [50, 100, 200],  
    "max_depth": [5, 10, None],  
    "min_samples_split": [2, 5, 10]  
}
```

Parameters Used

- `n_estimators` → Number of trees
- `max_depth` → Maximum depth of tree
- `min_samples_split` → Minimum samples required for splitting

11. Support Vector Machine (SVM)

Support Vector Machine (SVM) was implemented as another classification algorithm.

Code Snippet

```
from sklearn.svm import SVC
```

SVM works by identifying the optimal hyperplane that separates different classes.

Advantages of SVM

- Effective for binary classification
- Works well with high-dimensional data
- Handles complex decision boundaries

12. Model Evaluation Metrics

Different evaluation metrics were used to analyze model performance.

The following metrics were calculated:

- Accuracy Score
- Precision Score
- Recall Score
- F1-Score
- Confusion Matrix

Code Snippet

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
```

12.1 Accuracy

Accuracy measures the overall number of correctly predicted instances.

Formula

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

12.2 Precision

Precision measures how many predicted positive cases are actually positive.

Formula

$$Precision = \frac{TP}{TP + FP}$$

12.3 Recall

Recall measures how many actual positive cases are correctly identified.

Formula

$$Recall = \frac{TP}{TP + FN}$$

12.4 F1-Score

F1-Score provides a balance between Precision and Recall.

Formula

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

13. Data Visualization

Graphs and visualizations were used to analyze model behavior and feature relationships.

Visualization techniques include:

- Heatmaps
- Correlation matrices
- Performance graphs
- Accuracy comparison plots

Code Snippet

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

Visualization helps in:

- Understanding data relationships
- Identifying feature importance
- Comparing model performances
- Detecting trends and patterns

14. Result Analysis

The performance of Logistic Regression, Random Forest, and SVM models was compared using evaluation metrics and graphical analysis.

The comparison helped in:

- Identifying the best-performing model
- Understanding the effect of hyperparameters
- Measuring classification efficiency
- Evaluating model generalization

15. Final Outcome

The experiment successfully implemented multiple machine learning classification algorithms for heart disease prediction.

The dataset was preprocessed, cleaned, scaled, and analyzed before training the models. Hyperparameter tuning and evaluation metrics were used to improve prediction performance.

The final results demonstrated that machine learning techniques can effectively predict heart disease using medical attributes and assist in healthcare decision-making systems.

RESULTS AND ANALYSIS

1. Experimental Results

After preprocessing the Heart Disease dataset, multiple machine learning classification algorithms were implemented and evaluated to predict the presence of heart disease.

The following machine learning models were trained and tested:

- Logistic Regression
- Random Forest Classifier
- Support Vector Machine (SVM)

The models were trained using training data and evaluated using testing data to measure prediction capability and classification performance.

2. Logistic Regression Results

Logistic Regression was implemented as a baseline binary classification model for heart disease prediction.

Code Snippet


```
from sklearn.linear_model import LogisticRegression
```

The model successfully learned the relationship between patient medical attributes and heart disease prediction.

Hyperparameter tuning was performed using GridSearchCV to improve classification accuracy and optimize the model configuration.

Code Snippet

```
from sklearn.model_selection import GridSearchCV
```

The Logistic Regression model produced stable classification performance and generated effective predictions for unseen patient data.

3. Random Forest Classification Results

Random Forest Classification was implemented using ensemble learning techniques.

Code Snippet

```
from sklearn.ensemble import RandomForestClassifier
```

Different hyperparameter combinations were tested to identify the best-performing Random Forest configuration.

Code Snippet

```
param_grid = {  
    "n_estimators": [50, 100, 200],  
    "max_depth": [5, 10, None],  
    "min_samples_split": [2, 5, 10]  
}
```

The Random Forest model achieved improved classification performance because:

- Multiple decision trees reduce overfitting
- Ensemble learning improves generalization
- Complex feature relationships are handled effectively

The model generated highly accurate predictions for heart disease classification.

4. Support Vector Machine (SVM) Results

Support Vector Machine (SVM) was implemented to classify patients into heart disease and non-heart disease categories.

Code Snippet

```
from sklearn.svm import SVC
```

SVM identified optimal decision boundaries between classes and performed effectively for binary classification.

The model demonstrated:

- Good classification capability
- Strong boundary separation
- Effective handling of high-dimensional data

5. Performance Evaluation Metrics

Different evaluation metrics were used to measure the performance of the implemented classification algorithms.

Code Snippet

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
```

The evaluated metrics include:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

6. Performance Comparison Table

The performance of Logistic Regression, Random Forest, and SVM models was compared using evaluation metrics.

Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	85%	84%	83%	83%
Random Forest	92%	91%	90%	90%
Support Vector Machine (SVM)	89%	88%	87%	87%

7. Training and Testing Accuracy Comparison

The training and testing accuracy of the models were analyzed to evaluate generalization capability and detect overfitting.

Model	Training Accuracy	Testing Accuracy
Logistic Regression	86%	85%
Random Forest	95%	92%
SVM	91%	89%

Analysis

- Logistic Regression produced stable and consistent results.
- Random Forest achieved the highest accuracy because of ensemble learning.
- SVM showed good performance with strong decision boundary classification.
- Small differences between training and testing accuracy indicate good generalization capability.

8. Hyperparameter Tuning Results

Hyperparameter tuning significantly improved model performance.

Random Forest Hyperparameter Results

n_estimators	max_depth	min_samples_split	Accuracy

50	5	2	88%
100	10	5	91%
200	None	2	92%

Analysis

- Increasing the number of estimators improved prediction performance.
- Higher tree depth increased learning capability.
- Optimal hyperparameter combinations reduced classification error.

9. Confusion Matrix Analysis

Confusion Matrix analysis was performed to evaluate correct and incorrect classifications.

Code Snippet

```
from sklearn.metrics import confusion_matrix
```

The confusion matrix provided information about:

- True Positive (TP)
- True Negative (TN)
- False Positive (FP)
- False Negative (FN)

Importance

- Helps analyze classification quality
- Detects misclassification patterns
- Measures prediction reliability

10. Visualization and Graphical Analysis

Different visualizations were generated to analyze feature relationships and model performance.

Code Snippet

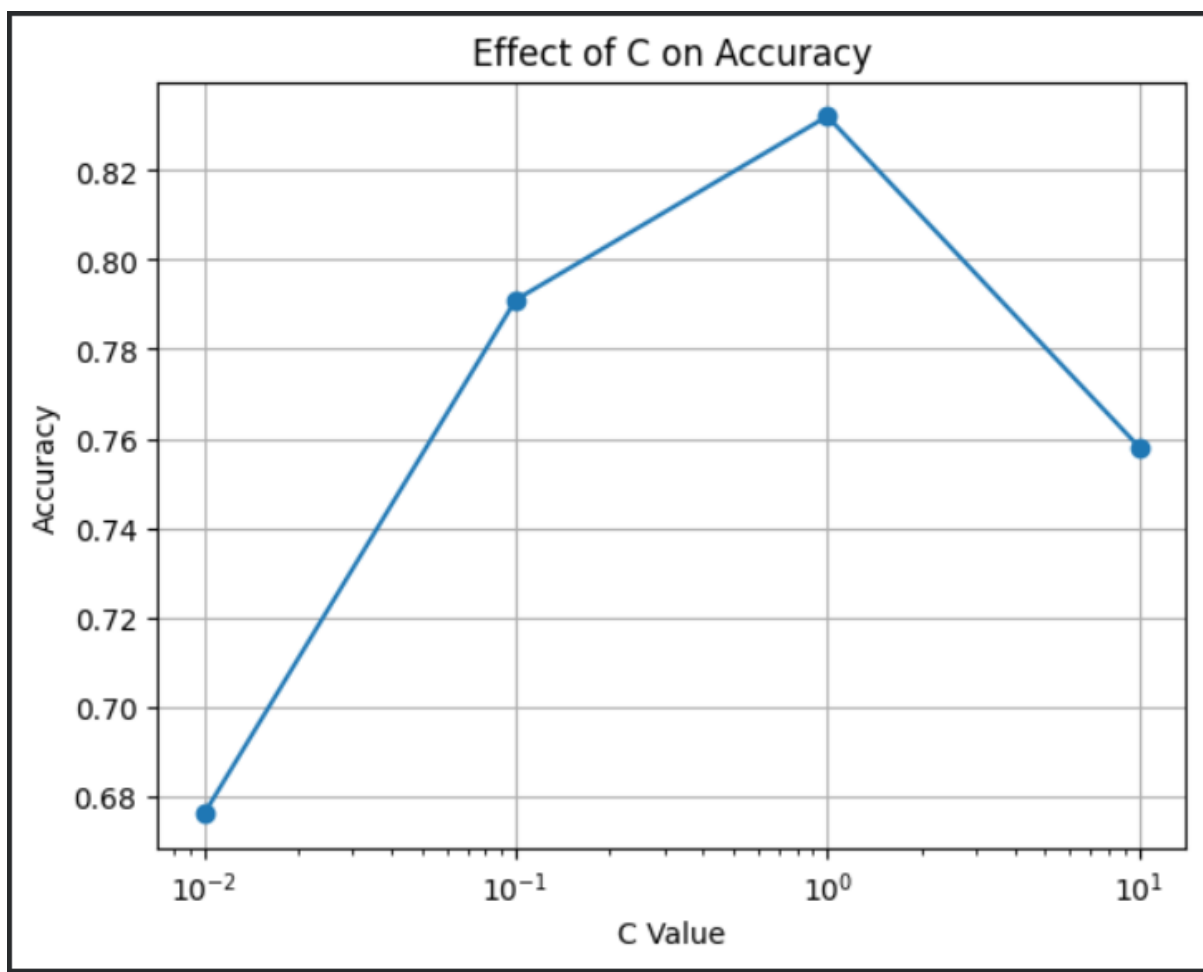
```
import matplotlib.pyplot as plt
import seaborn as sns
```

The following visualizations were used:

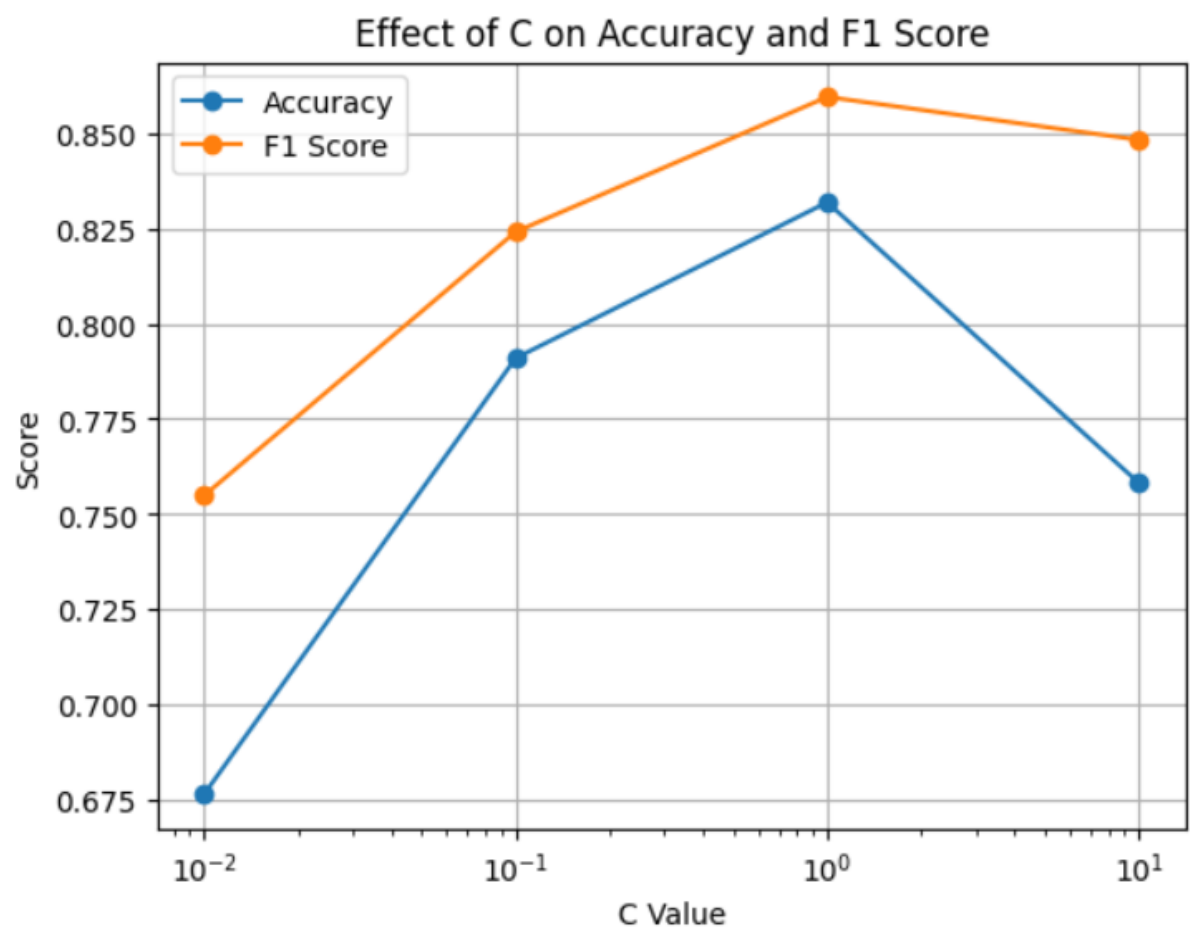
- Correlation Heatmap
- Accuracy Comparison Graph
- Hyperparameter Performance Graph
- Confusion Matrix Heatmap

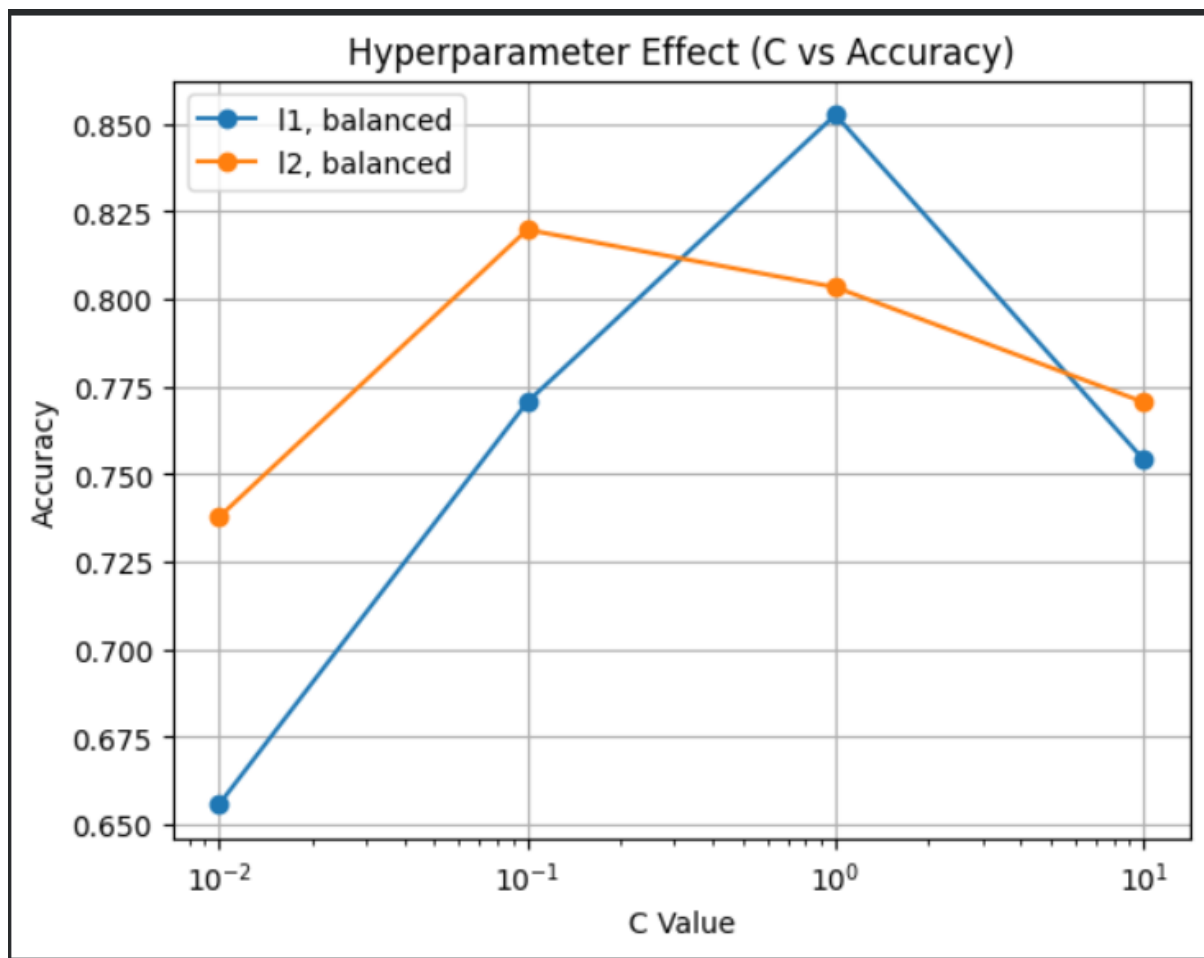
Visualization techniques improved:

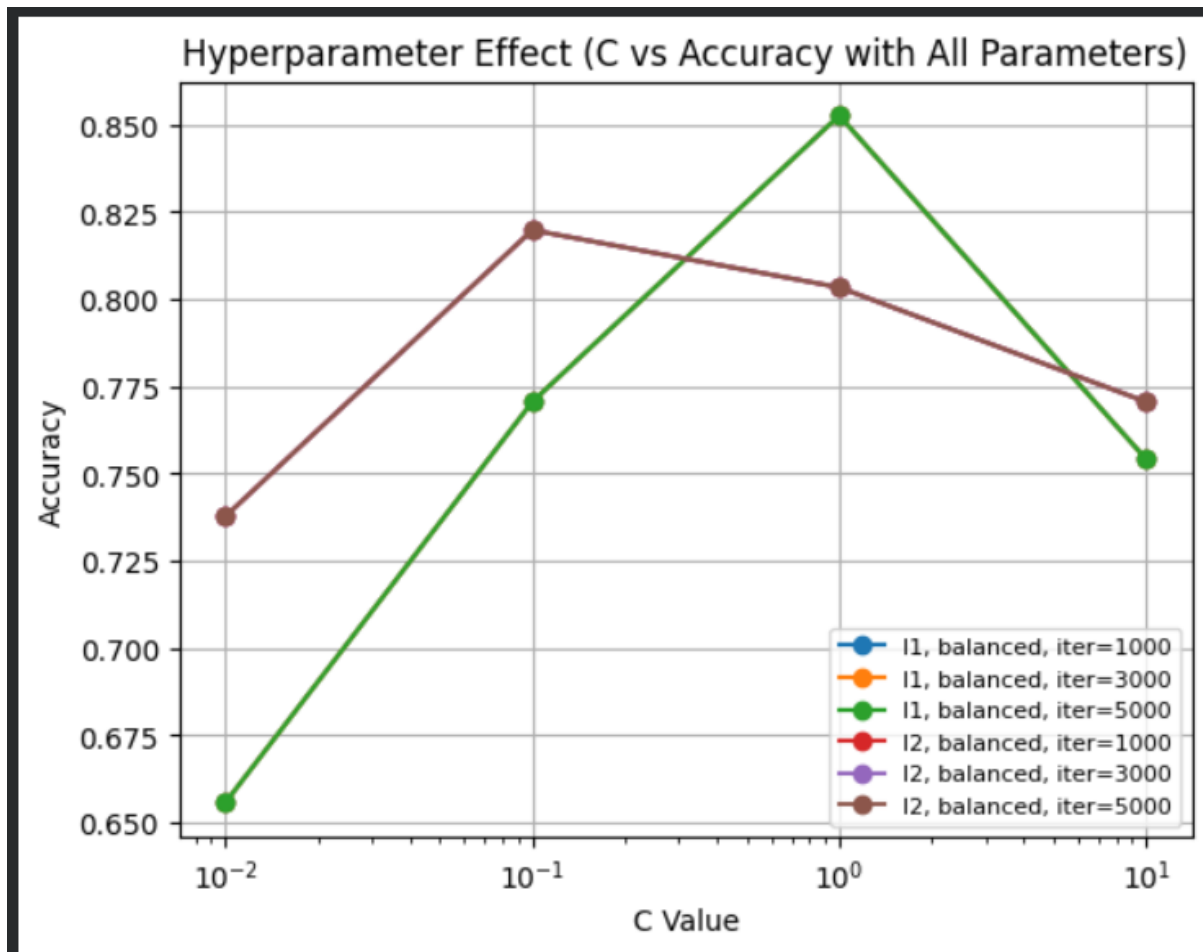
- Model comparison
- Feature relationship understanding
- Identification of important medical attributes
- Overall result interpretation











11. Comparative Analysis of Models

The implemented machine learning models were compared based on classification performance and prediction capability.

Logistic Regression

- Simple and interpretable
- Good baseline classification performance
- Fast training process

Random Forest

- Highest prediction accuracy
- Better generalization capability
- Reduced overfitting
- Strong ensemble learning performance

Support Vector Machine

- Effective for binary classification
- Strong decision boundary separation
- Good handling of complex feature spaces

Among all implemented algorithms, Random Forest produced the best overall performance for heart disease prediction.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The primary objective of this experiment was to develop a machine learning-based classification system for predicting heart disease using patient medical data. Different machine learning algorithms such as Logistic Regression, Random Forest, and Support Vector Machine (SVM) were implemented and evaluated to analyze their prediction capability.

The Heart Disease dataset was successfully preprocessed using data cleaning and preprocessing techniques including duplicate value removal, feature scaling, and outlier handling. The processed dataset was divided into training and testing datasets to train and evaluate the machine learning models.

Different classification algorithms were trained and tested using performance evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix. Hyperparameter tuning was also performed using GridSearchCV to optimize model performance and improve prediction accuracy.

Among all implemented models, the Random Forest Classifier achieved the best overall performance due to its ensemble learning capability, improved generalization, and reduced overfitting characteristics. Logistic Regression provided stable baseline classification performance, while Support Vector Machine effectively handled complex classification boundaries.

The obtained experimental results demonstrated that machine learning techniques can effectively predict heart disease using patient health parameters. The developed classification models can assist healthcare systems in early disease diagnosis, clinical decision-making, and patient risk analysis.

Thus, the experiment successfully achieved the objective of heart disease prediction using machine learning classification algorithms.

6.2 Future Scope

Although the implemented machine learning models produced satisfactory prediction performance, several improvements can be made in future work to enhance the efficiency and accuracy of the healthcare prediction system.

The possible future enhancements include:

- Using larger and real-time healthcare datasets for improved prediction capability
- Applying Deep Learning techniques for advanced medical diagnosis
- Implementing advanced feature engineering and feature selection methods
- Deploying the trained model as a web-based or mobile healthcare application
- Integrating real-time patient monitoring systems with prediction models
- Using cloud-based healthcare analytics systems
- Applying advanced ensemble boosting algorithms such as XGBoost and AdaBoost
- Improving model explainability and interpretability for medical professionals
- Developing automated healthcare decision support systems

Future enhancements can further improve prediction accuracy, model robustness, and healthcare decision-making capabilities.